# 252-0027

# Einführung in die Programmierung

## Übungen

# Woche 6: References, Recursion, Precondition

*Fabian Rohlik*
**Departement Informatik**
**ETH Zürich**

# Plan für heute

- Value vs Reference Wiederholung
- Rekursion
- Logisches Schliessen
- Vorbesprechung

# Value vs Reference

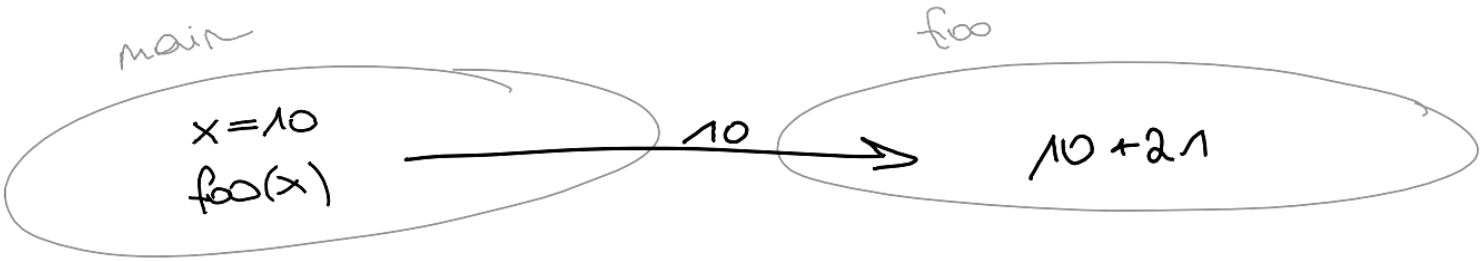
- Variablen enthalten entweder einen **Wert** (Value) oder eine **Referenz** (Reference).
- Variablen enthalten **nie** Objekte.
- Referenzen zeigen auf das Objekt im Speicher.

```java
public static void main(String[] args) {
    int x = 10;
    foo(x);
    System.out.println(x);
}

public static void foo(int x) {
    x = x+21;
}
```

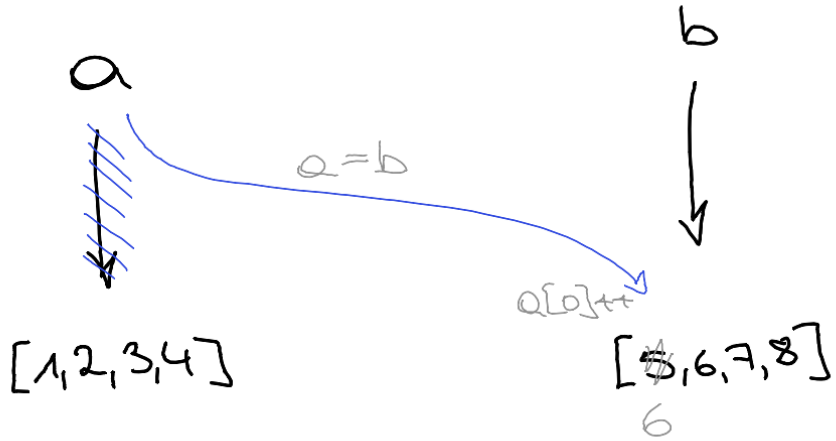
```java
public static void main(String[] args) {
    int x = 10;
    foo(x);
    System.out.println(x);
}

public static void foo(int x) {
    x = x+21;
}
```



```
int[] a = {1, 2, 3, 4};
int[] b = {5, 6, 7, 8};
a = b;
a[0]++;
System.out.println(Arrays.toString(b));
```

```
int[] a = {1, 2, 3, 4};
int[] b = {5, 6, 7, 8};
a = b;
a[0]++;
System.out.println(Arrays.toString(b));
```

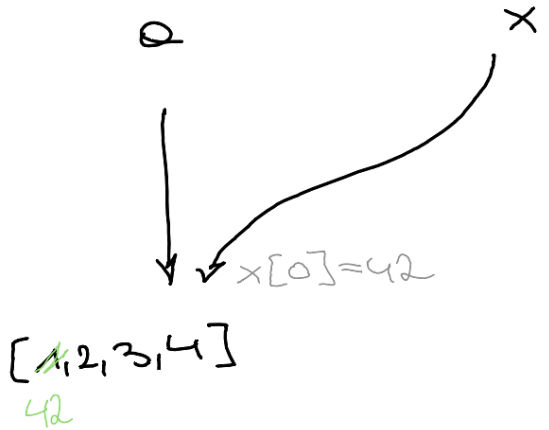


```java
public static void main(String[] args) {
    int[] a = {1, 2, 3, 4};
    foo(a);
    System.out.println(Arrays.toString(a))
}

public static void foo(int[] x) {
    x[0] = 42;
}
```

```
public static void main(String[] args) {
    int[] a = {1, 2, 3, 4};
    foo(a);
    System.out.println(Arrays.toString(a))
}

public static void foo(int[] x) {
    x[0] = 42;
}
```

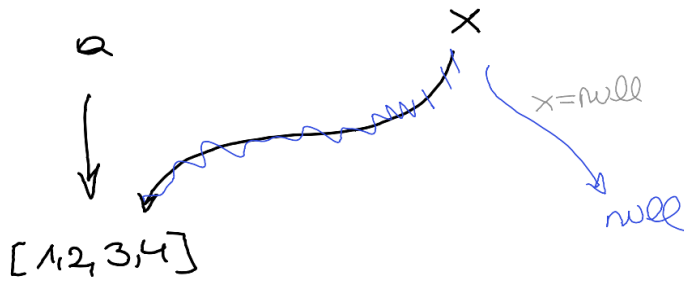


```
public static void main(String[] args) {
    int[] a = {1, 2, 3, 4};
    foo(a);
    System.out.println(Arrays.toString(a));
}

public static void foo(int[] x) {
    x = null;
}
```

```java
public static void main(String[] args) {
    int[] a = {1, 2, 3, 4};
    foo(a);
    System.out.println(Arrays.toString(a));
}

public static void foo(int[] x) {
    x = null;
}
```

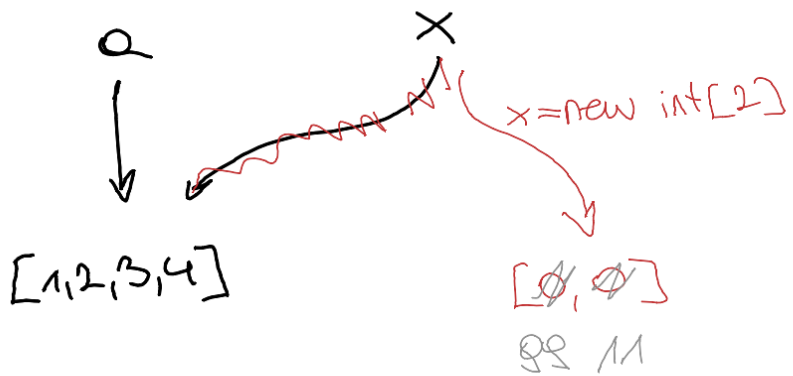


```
public static void main(String[] args) {
    int[] a = {1, 2, 3, 4};
    foo(a);
    System.out.println(Arrays.toString(a));
}

public static void foo(int[] x) {
    x = new int[2];
    x[0] = 99;
    x[1] = 11;
}
```

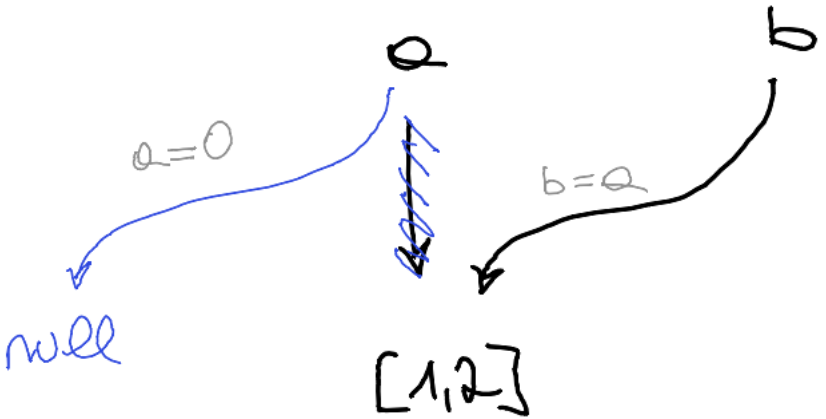
```java
public static void main(String[] args) {
    int[] a = {1, 2, 3, 4};
    foo(a);
    System.out.println(Arrays.toString(a));
}

public static void foo(int[] x) {
    x = new int[2];
    x[0] = 99;
    x[1] = 11;
}
```



```java
int[] a = {1, 2};
int[] b = a;
a = null;
System.out.println(b[0]);
```

```java
int[] a = {1, 2};
int[] b = a;
a = null;
System.out.println(b[0]);
```



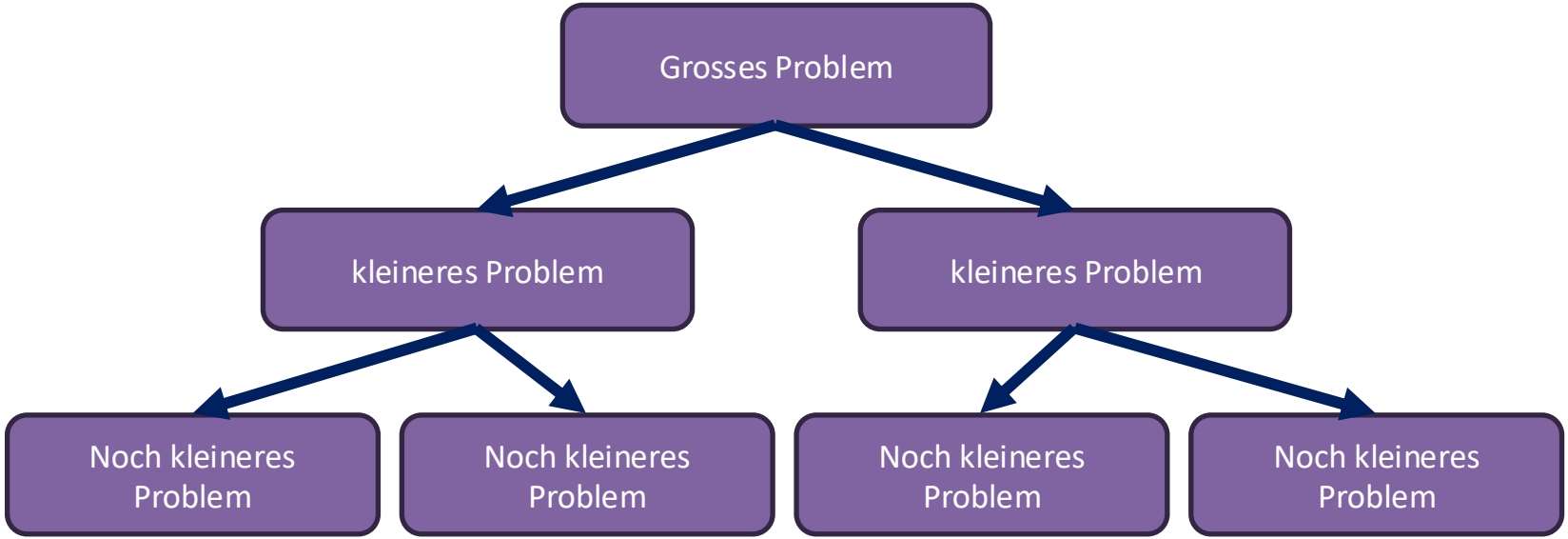
# Rekursion

Divide and Conquer

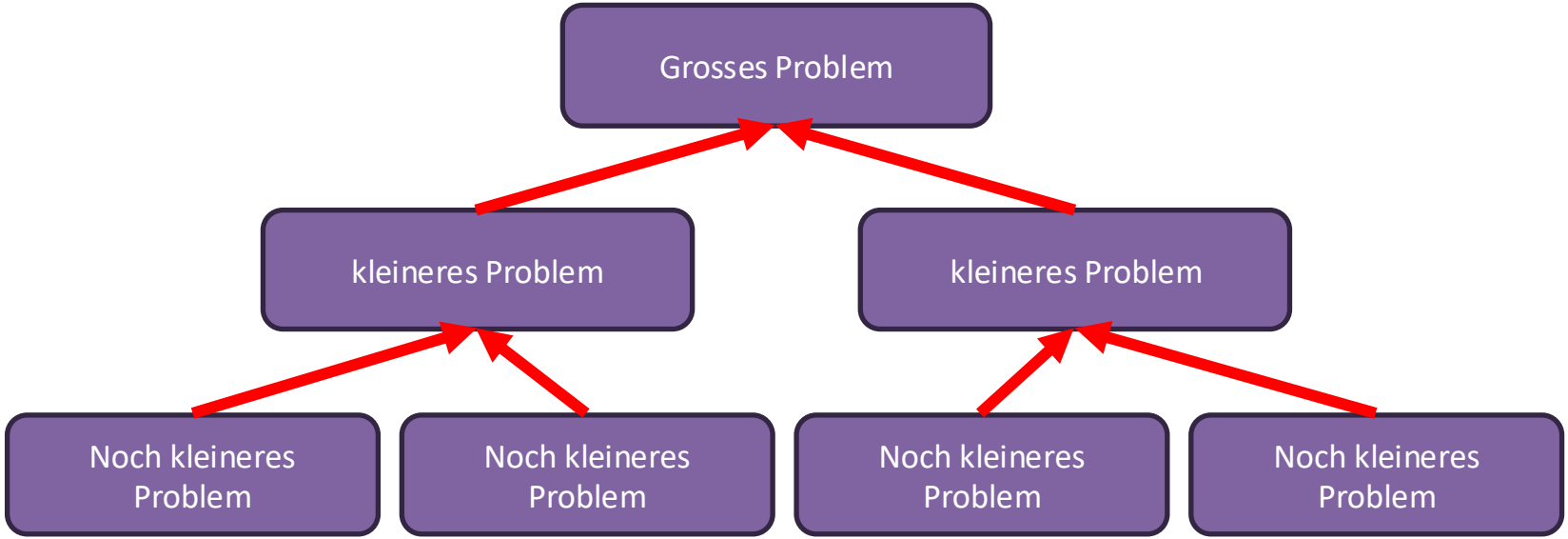
Decrease and Conquer

# Divide and Conquer

# Rekusion – Grundprinzip Divide and Conquer



# Rekusion – Grundprinzip Divide and Conquer



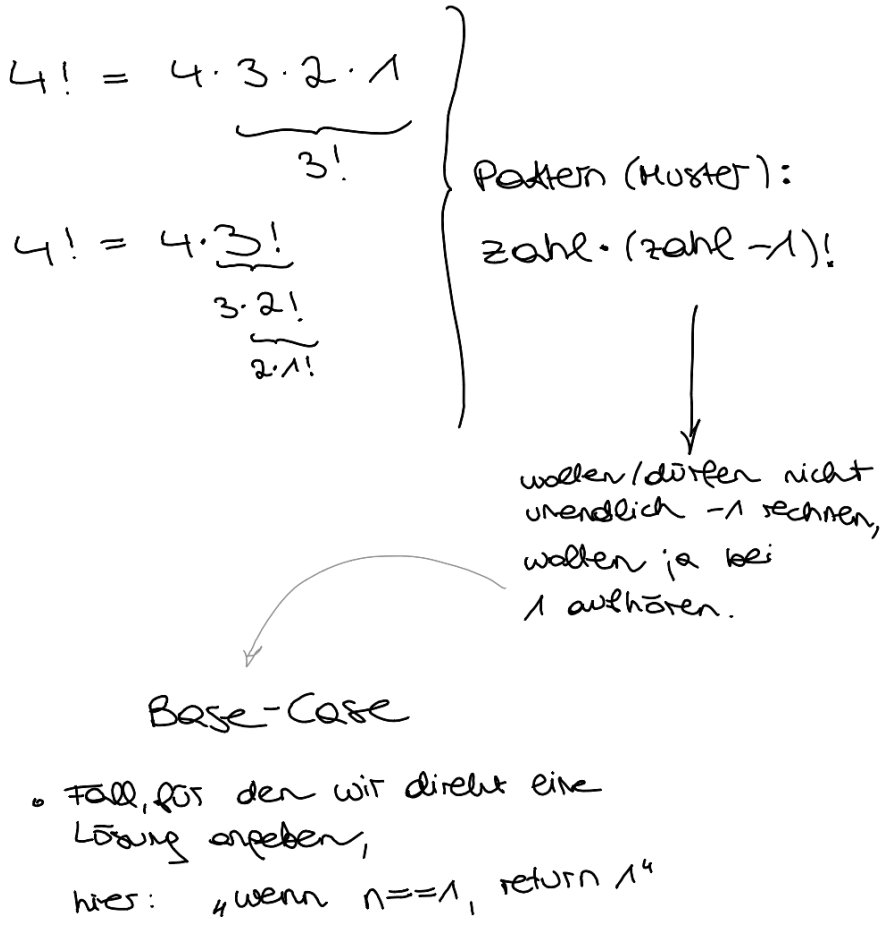
# Decrease and Conquer

# 1. Beispiel

# Fakultät berechnen

# Fakultät Überlegungen

# Beispiel Fakultät



# Decrease-and-Conquer - Fakultät

```
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```

factorial(3)

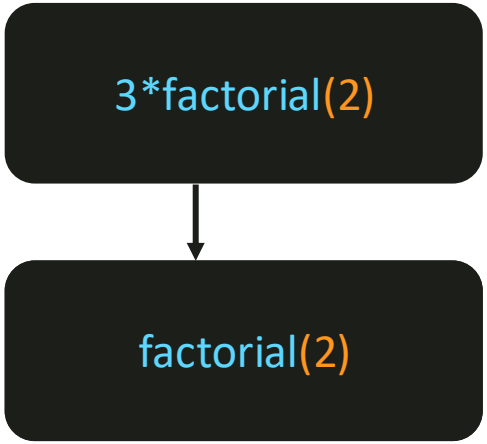
# Decrease-and-Conquer - Fakultät

```
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```

3*factorial(2)

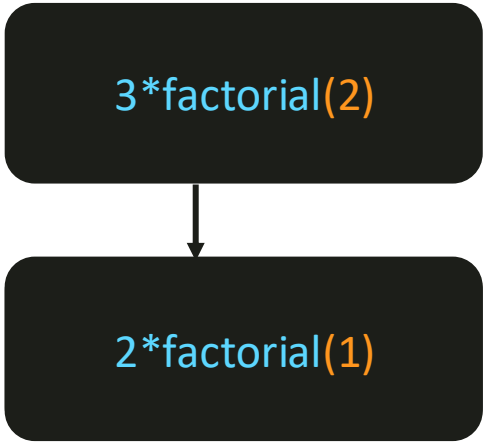
# Decrease-and-Conquer - Fakultät

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



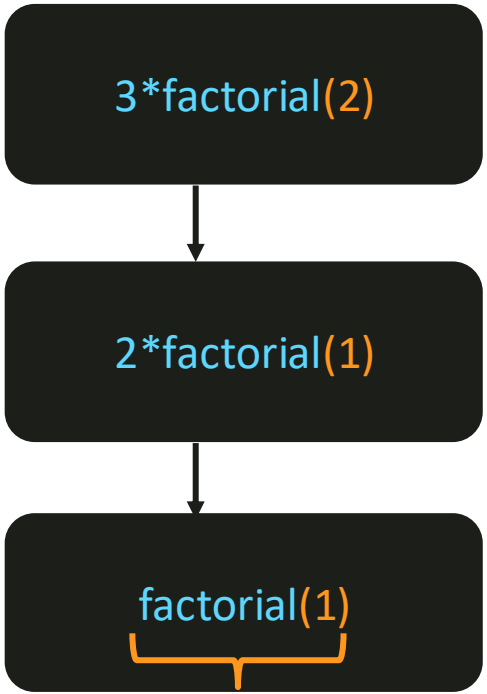
# Decrease-and-Conquer - Fakultät

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



# Decrease-and-Conquer - Fakultät

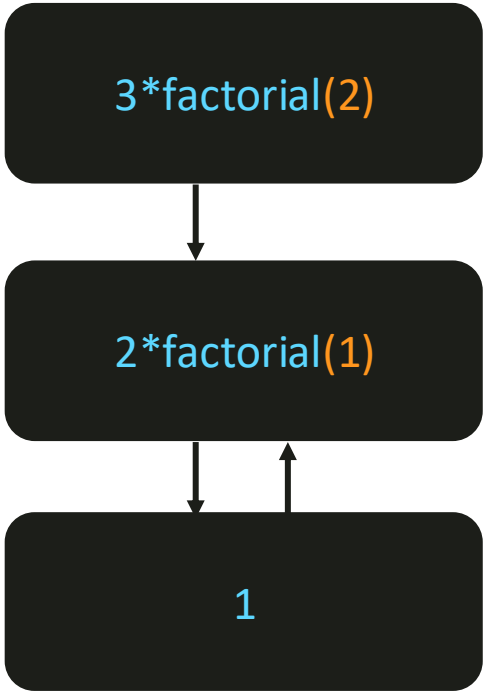
```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```




**Base Case!**

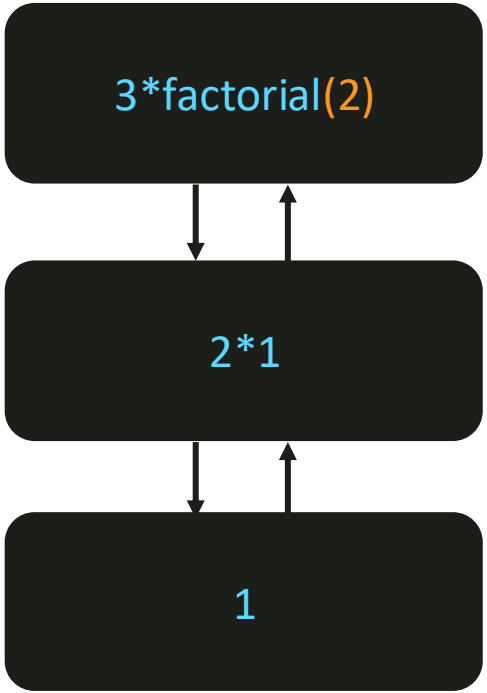
# Decrease-and-Conquer - Fakultät

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



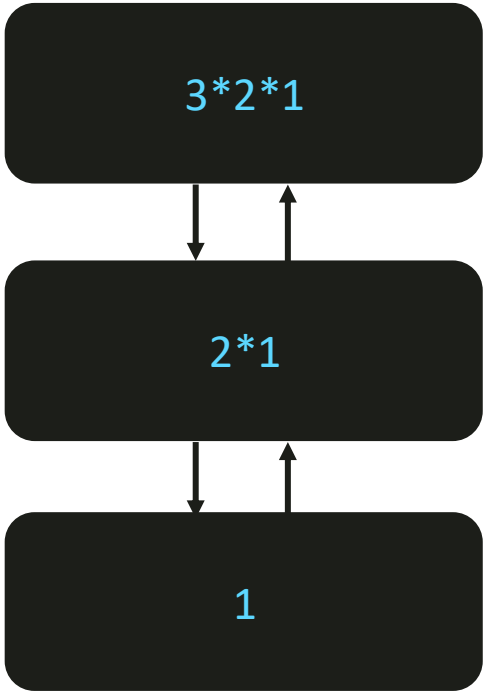
# Decrease-and-Conquer - Fakultät

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



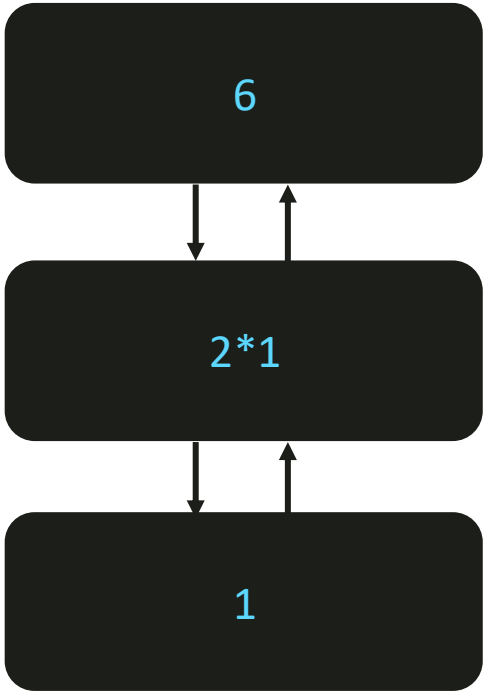
# Decrease-and-Conquer - Fakultät

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



# Decrease-and-Conquer - Fakultät

```java
public int factorial(int n) {
    if (n==1)return 1;
    return n*factorial(n-1);
}
```



# 2. Beispiel

# sum()

# Problem, das wir lösen wollen

**Aufgabe**: Schreiben Sie eine Funktion `sum(n)`, die gegeben eine nichtnegative Zahl, alle nichtnegativen Zahlen bis einschliesslich n aufsummiert.

**Beispiele**:

sum(0) = 0

sum(1) = 1 + 0

sum(4) = 4 + 3 + 2 + 1 + 0

Wir wollen dies nun **rekursiv** lösen.

# Überlegungen zu sum

# Überlegungen zu sum

Beispiel

- alle nichtnegativen Zahlen bis einschließlich n addieren

z.B. sum(4) = 4+3+2+1+0

1) Base-case, „was ist die einfachst-mögliche Eingabe / unser „Stopp" Fall"

sum(0) = 0

2) Finde ein Muster zur Vereinfachung („Reduktionsschritt")

- sum(4) = 4+3+2+1+0
  sum(3)

⇒ sum(4) = 4 + sum(3)

Allgemein: sum(n) = n + sum(n-1)

# Schritte

1. Was ist die einfachste mögliche Eingabe?
2. Beispiele ausprobieren und visualisieren
3. Leite größere Beispiele von kleineren Beispielen ab
4. Verallgemeinere das Muster

5. Schreibe den Code

# 1. Die einfachste Eingabe

- Hier ist es **n = 0: sum(0) = 0**
- das ist unser *Base Case,* also ein Fall, für den wir direkt eine Lösung angeben
- alle andere Lösungen werden rekursiv gelöst

# Schritte

1. Was ist die einfachste mögliche Eingabe?

→ Base Case



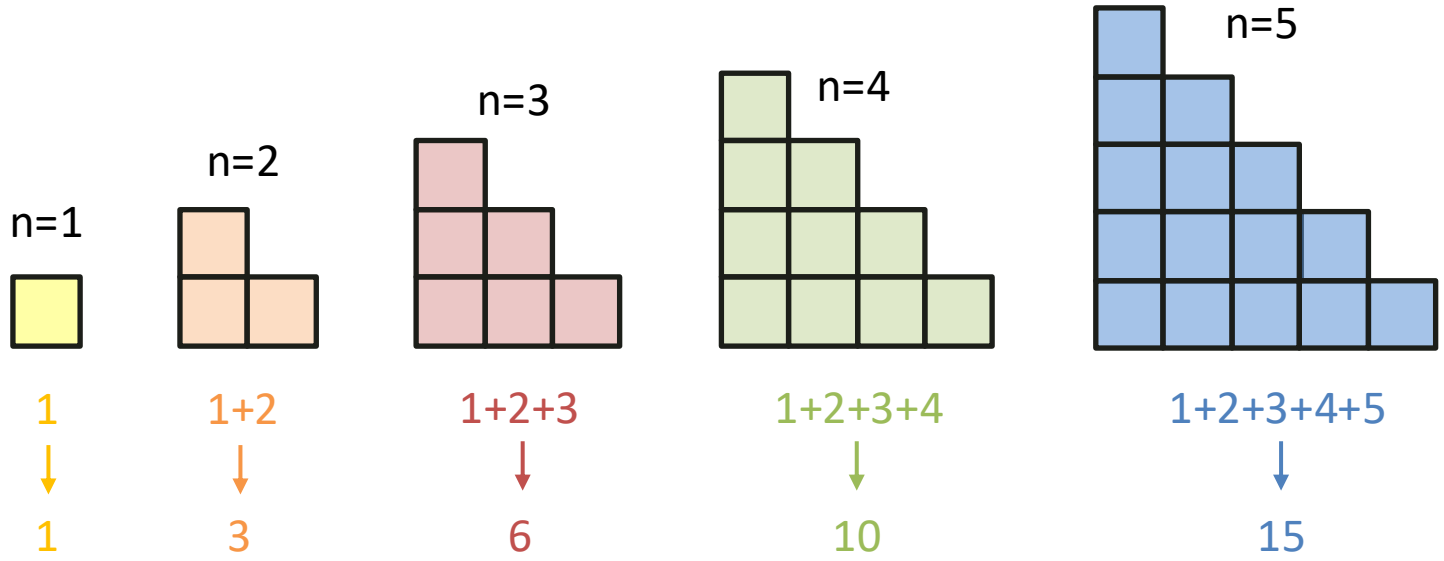
2. Beispiele ausprobieren und visualisieren
3. Leite größere Beispiele von kleineren Beispielen ab
4. Verallgemeinere das Muster

→ Rekursionsschritt

5. Schreibe den Code

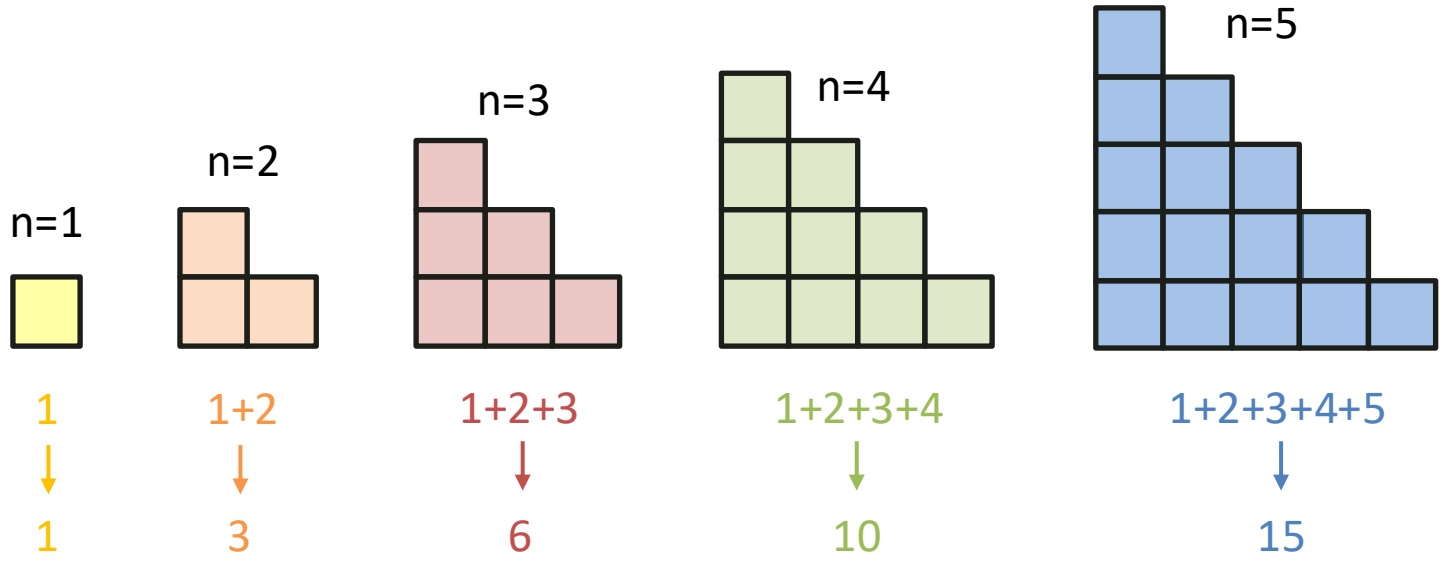
# 2. Die Beispiele

- Visualisiere, wie die Ein- und Ausgaben der Funktion miteinander zusammenhängen



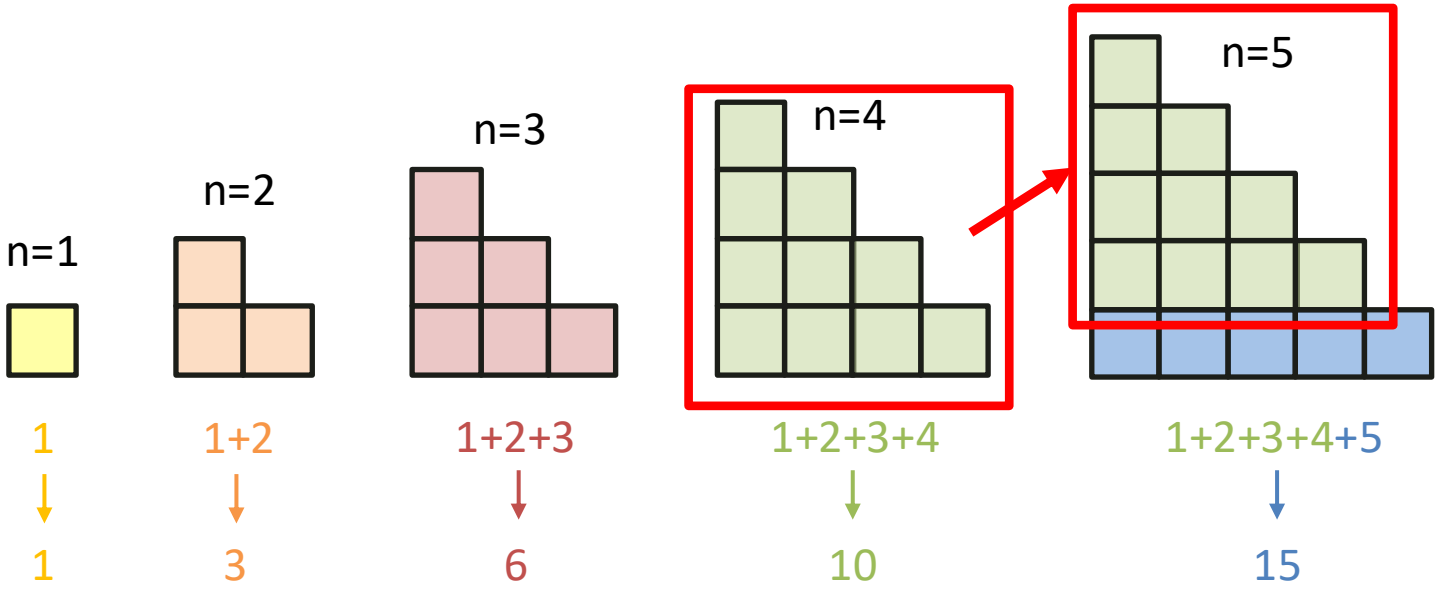
# 3. Erkenne den Zusammenhang

- „Wenn ich die Antwort für n=4 gegeben hätte, wie komme ich auf die Antwort für n=5? Für n=3 auf n=4? …“



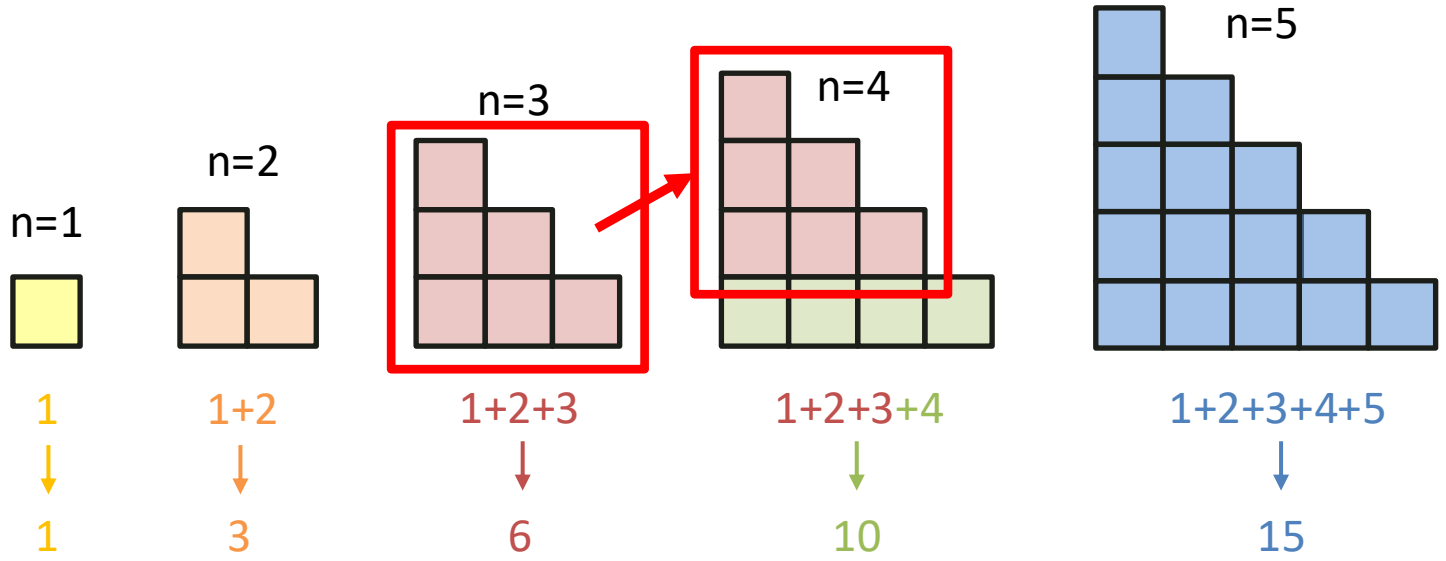
# 3. Erkenne den Zusammenhang

- „Wenn ich die Antwort für n=4 gegeben hätte, wie komme ich auf die Antwort für n=5? Für n=3 auf n=4? …“



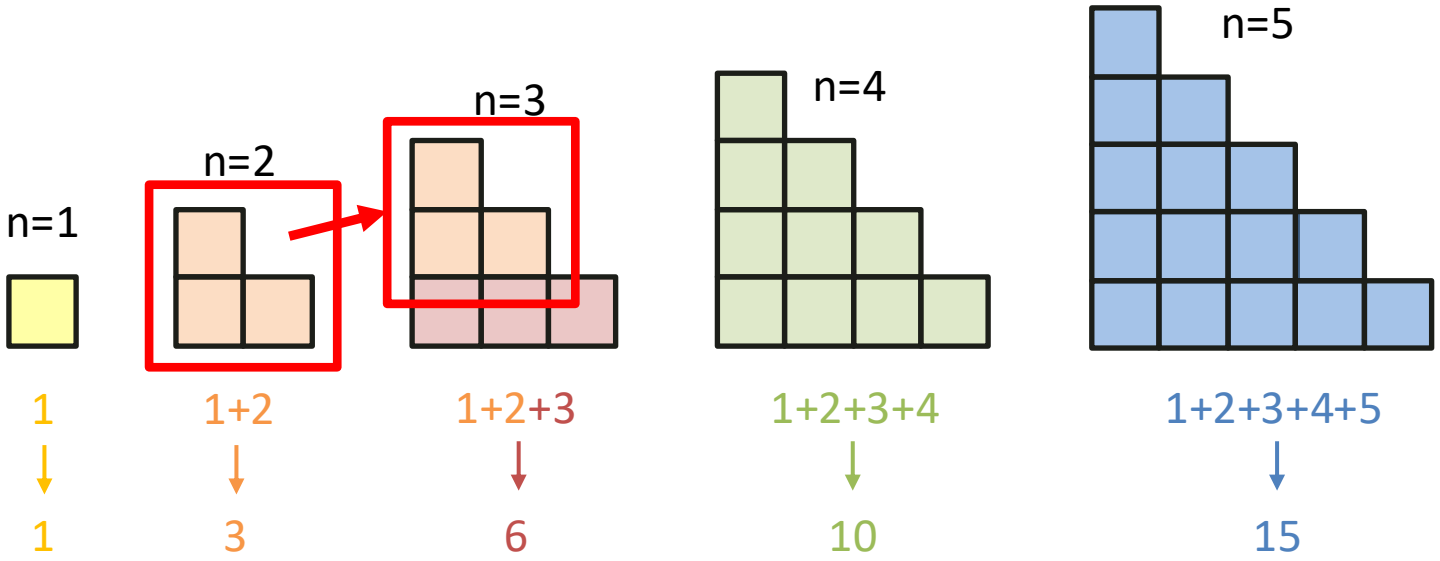
# 3. Erkenne den Zusammenhang

- „Wenn ich die Antwort für n=4 gegeben hätte, wie komme ich auf die Antwort für n=5? Für n=3 auf n=4? …“



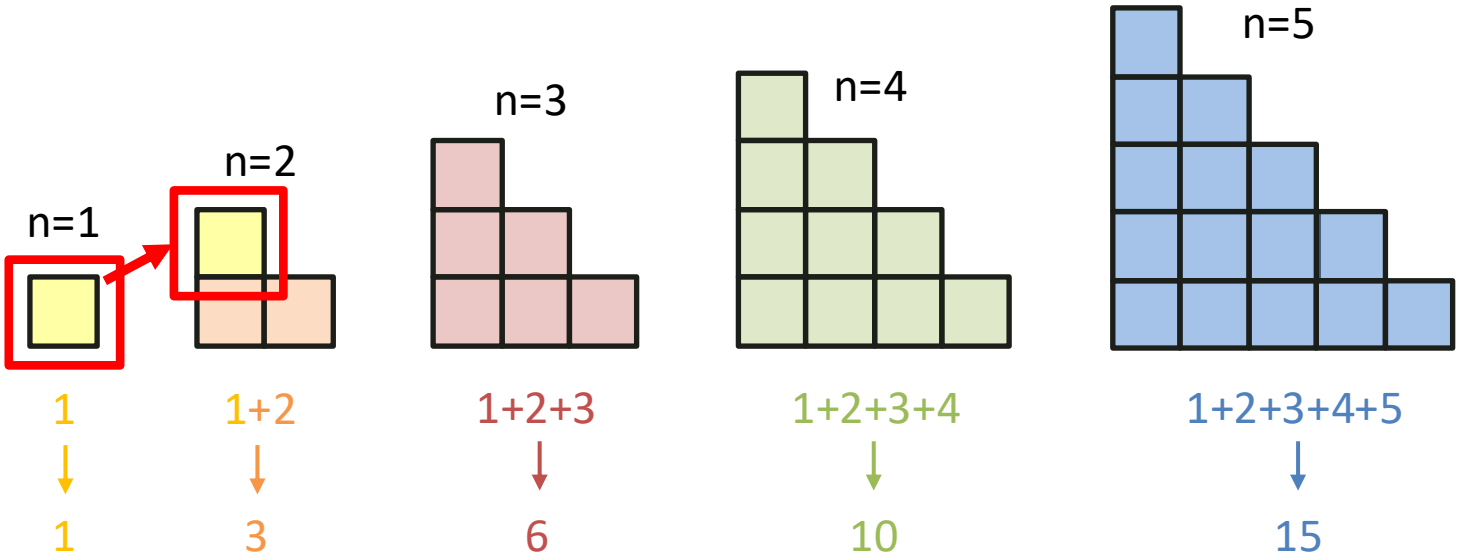
# 3. Erkenne den Zusammenhang

- „Wenn ich die Antwort für n=4 gegeben hätte, wie komme ich auf die Antwort für n=5? Für n=3 auf n=4? …“

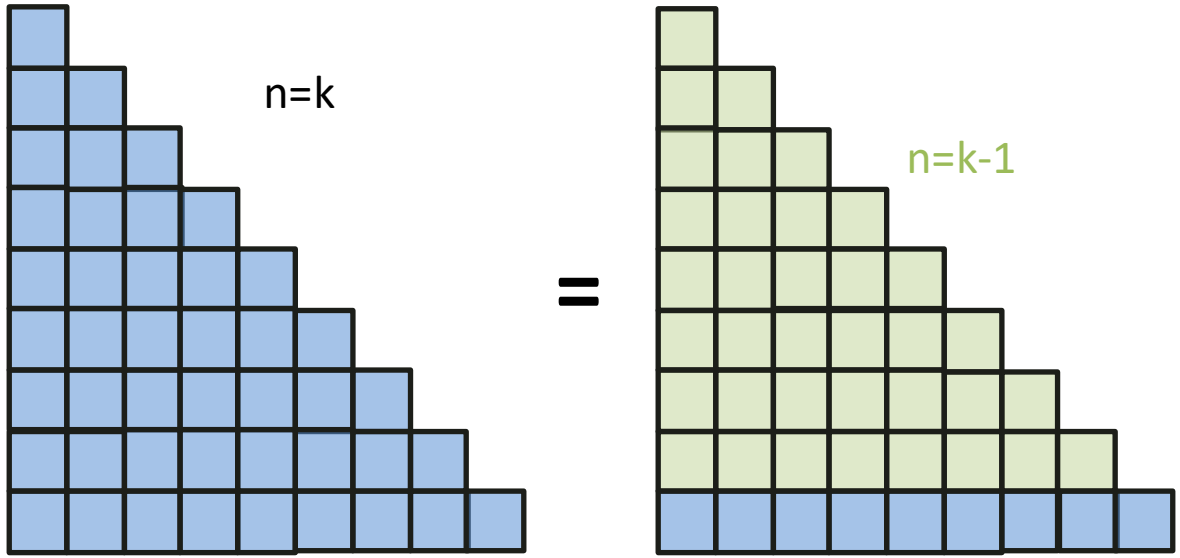


# 3. Erkenne den Zusammenhang

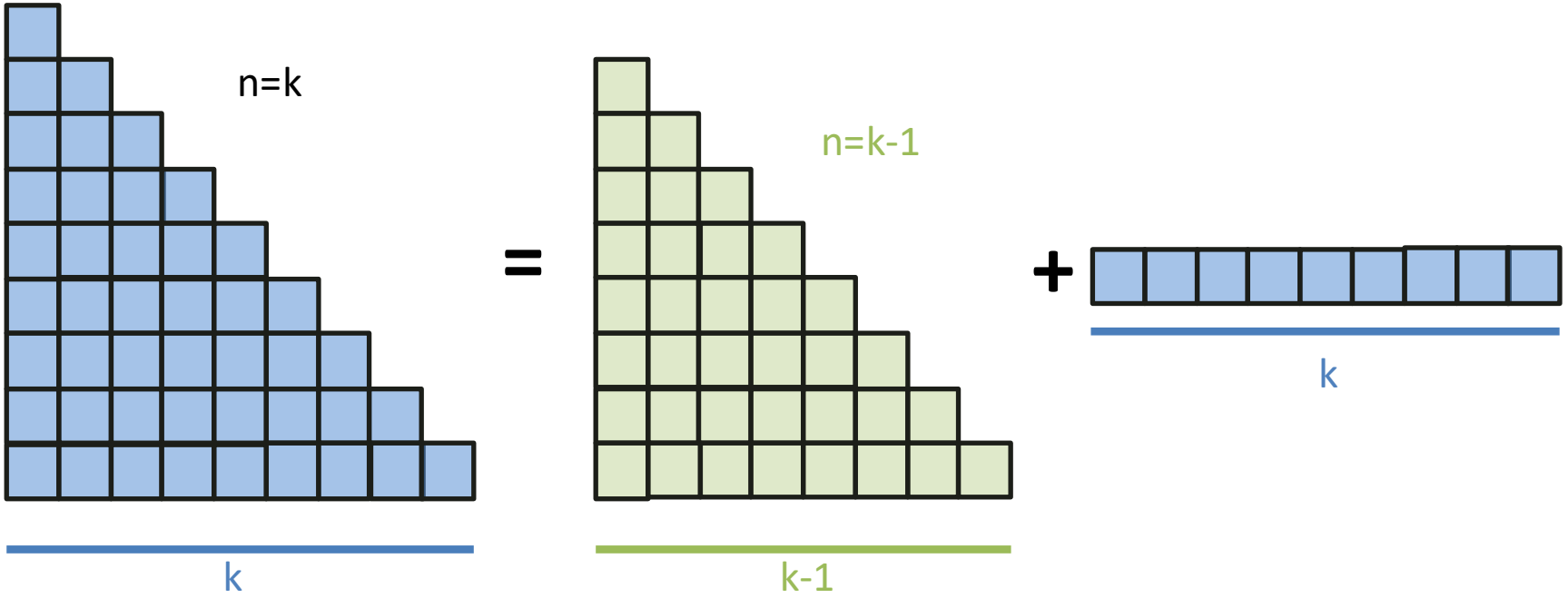
- „Wenn ich die Antwort für n=4 gegeben hätte, wie komme ich auf die Antwort für n=5? Für n=3 auf n=4? …“



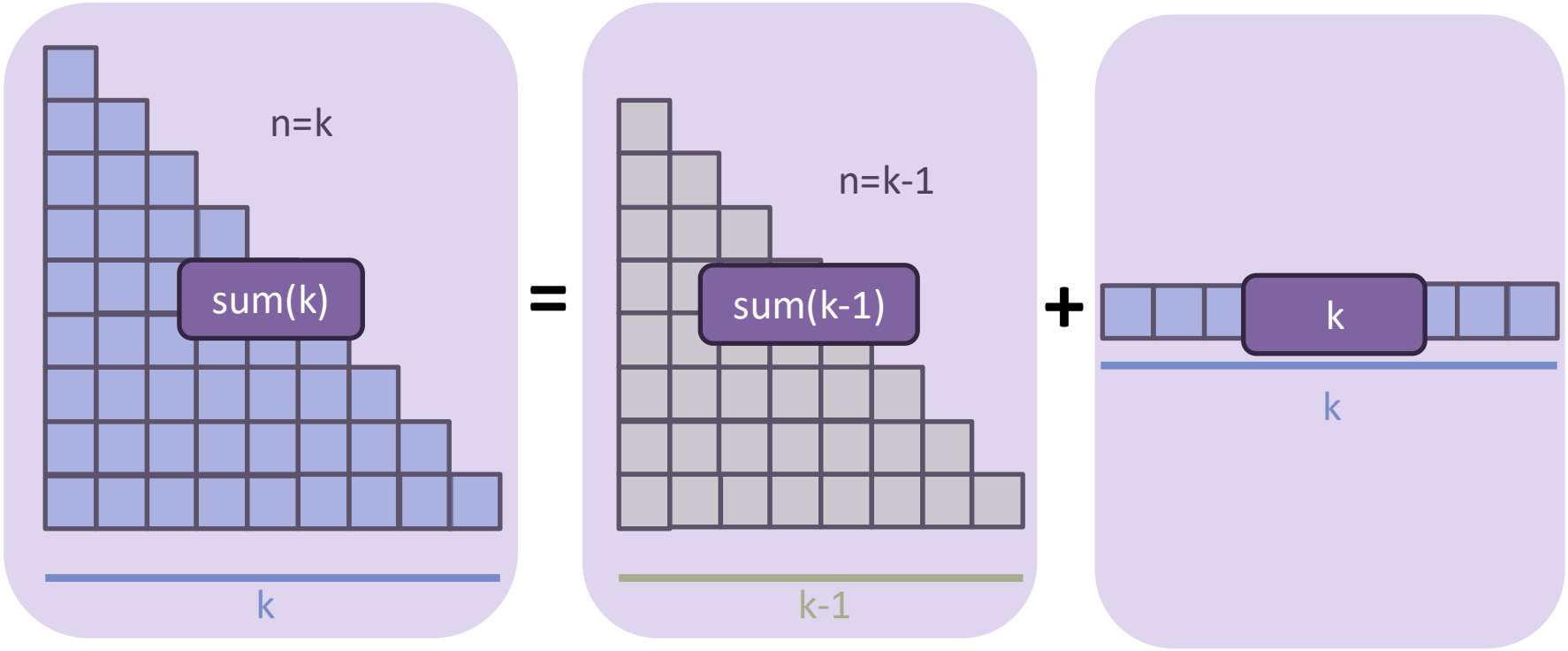
# 4. Das Muster



# 4. Das Muster



# 4. Das Muster



# Schritte

1. Was ist die einfachste mögliche Eingabe?

   → Base Case



2. Beispiele ausprobieren und visualisieren
3. Leite größere Beispiele von kleineren Beispielen ab
4. Verallgemeinere das Muster

   → Rekursionsschritt



5. Schreibe den Code

# 5. Der Code

Was haben wir herausgefunden?

Im 1. Schritt haben wir den *Base Case* gefunden:

**Falls n=0, dann sum(n) = 0**

Danach haben wir das *Rekursionsmuster* gefunden:

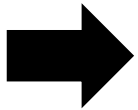
**sum(n) = sum(n-1)+n**

→ Dies können wir jetzt in Code umschreiben

# 5. Base Case + Muster → Code

Falls n=0: sum(n) = 0

Sonst: sum(n) = sum(n-1)+n



```
public static int sum(int n) {

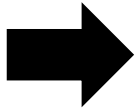


}
```

# 5. Base Case + Muster → Code

Falls n=0: sum(n) = 0

Sonst: sum(n) = sum(n-1)+n



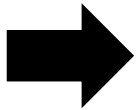
```
public static int sum(int n) {
    if (n == 0) {
        return 0;
    }

}
```

# 5. Base Case + Muster → Code

Falls n=0: sum(n) = 0

Sonst: sum(n) = sum(n-1)+n



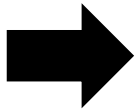
```
public static int sum(int n) {
    if (n == 0) {
        return 0;
    }

}
```

# 5. Base Case + Muster → Code

Falls n=0: sum(n) = 0

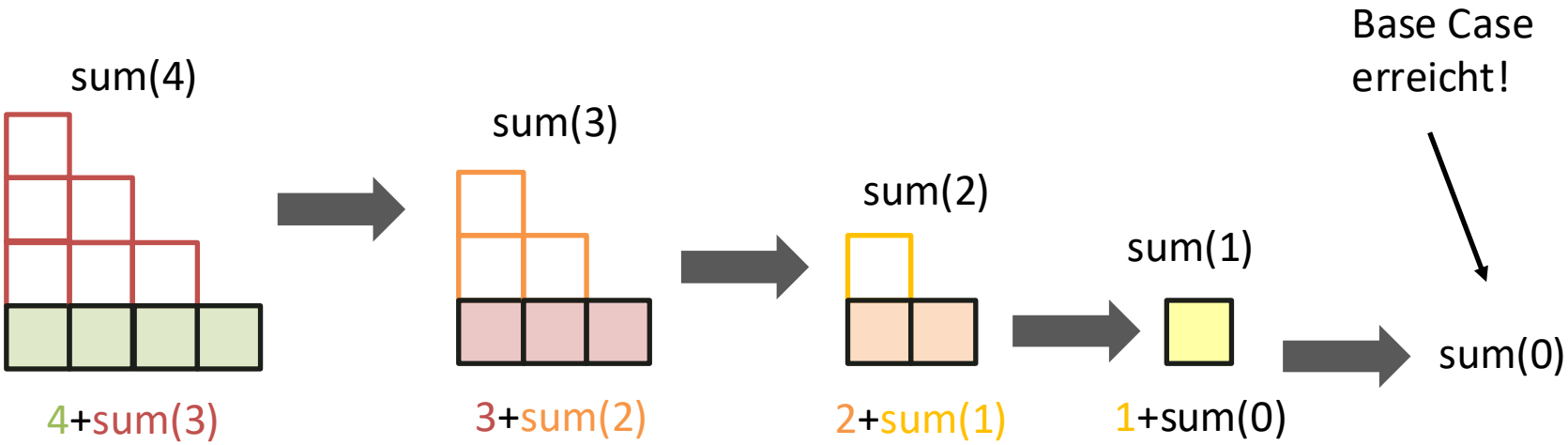
Sonst: sum(n) = sum(n-1)+n



```java
public static int sum(int n) {
    if (n == 0) {
        return 0;
    }
    return sum(n-1) + n;
}
```

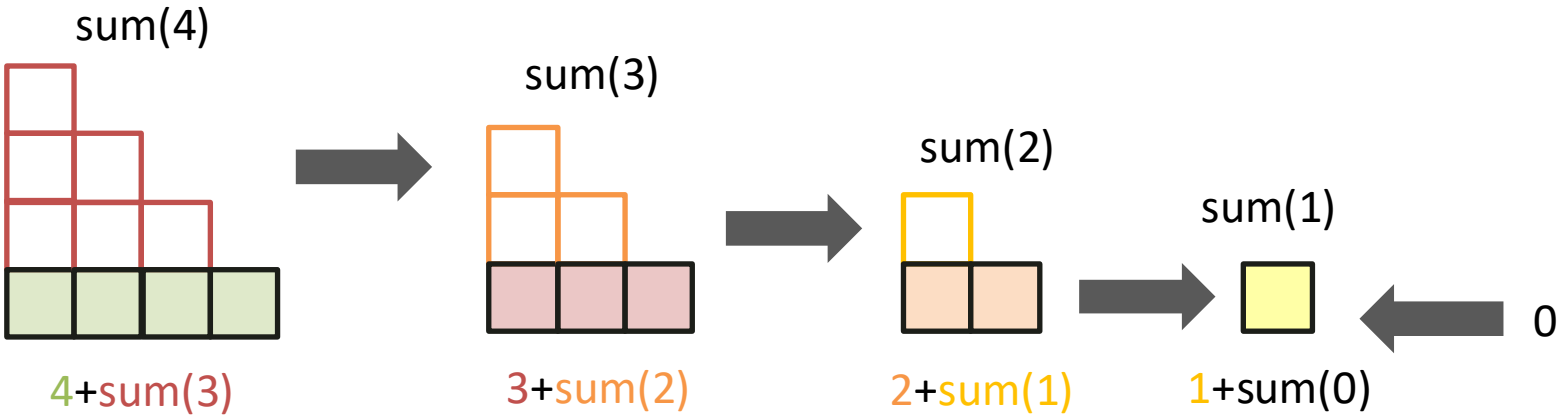
# Was passiert wenn ich diesen Code ausführe?

1. Rekursive Aufrufe



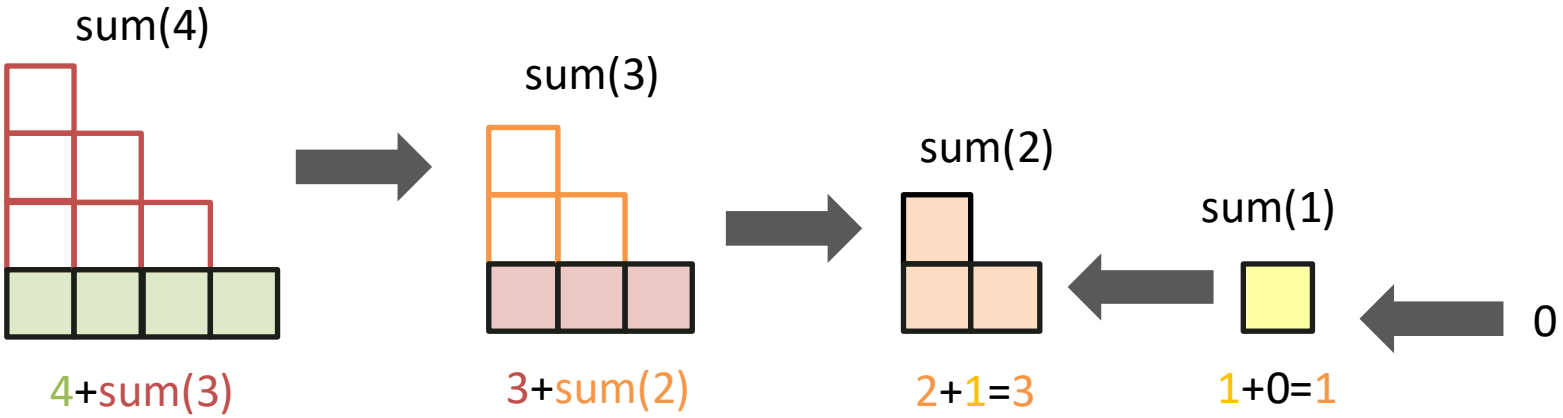
# Was passiert wenn ich diesen Code ausführe?

2. Auflösen



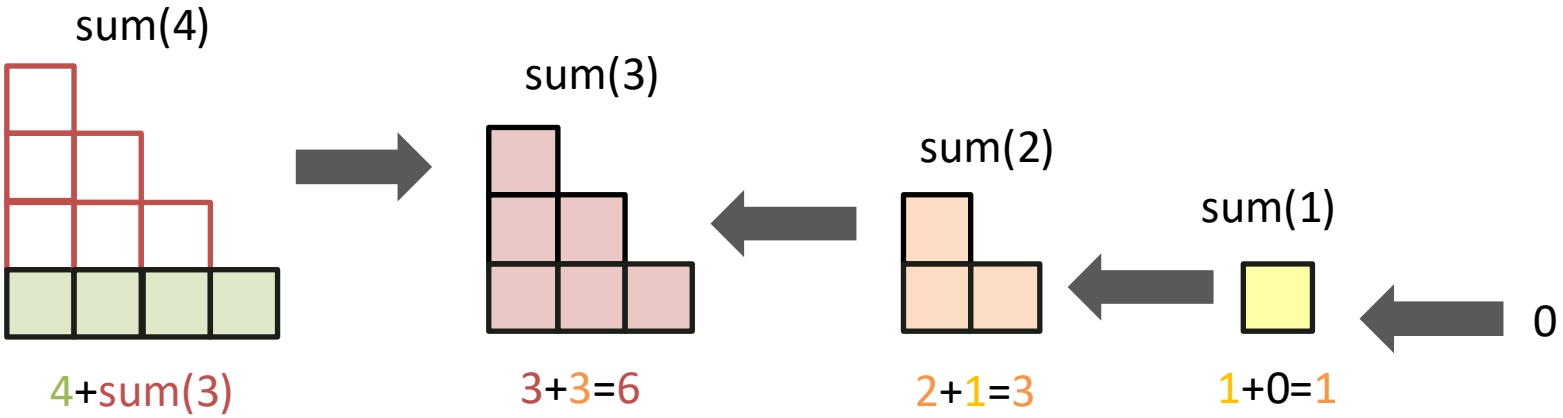
# Was passiert wenn ich diesen Code ausführe?

2. Auflösen



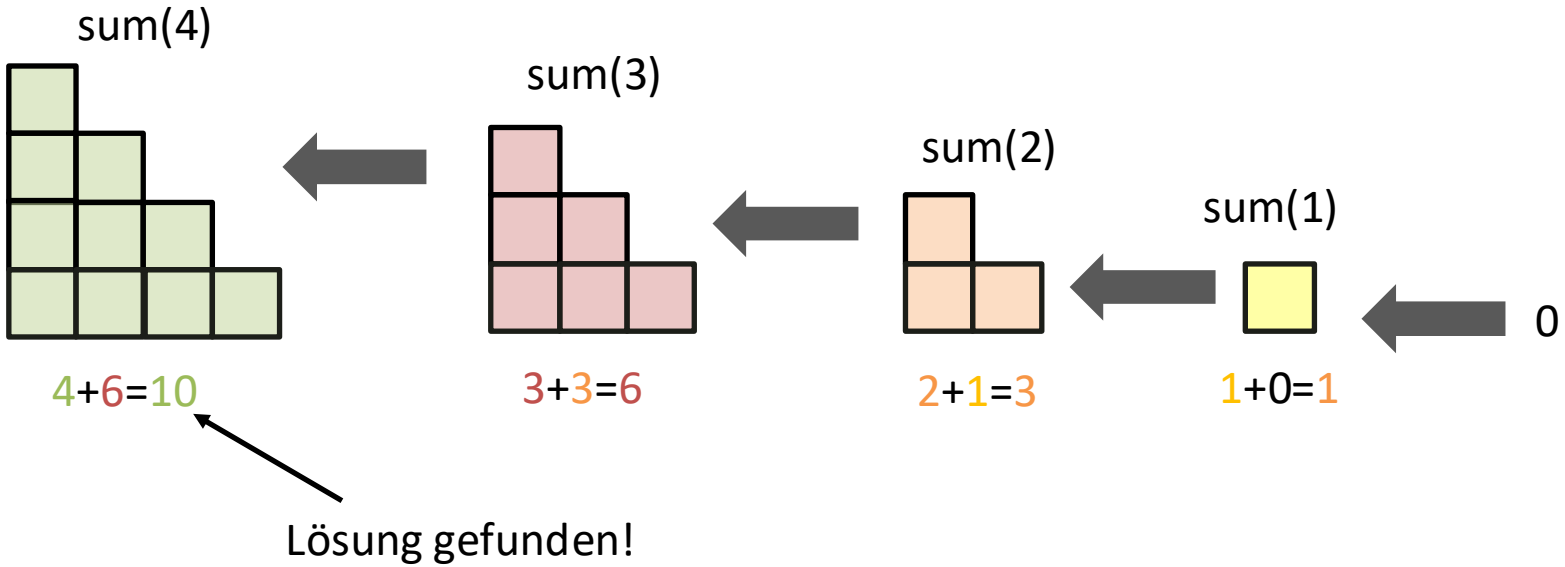
# Was passiert wenn ich diesen Code ausführe?

2. Auflösen



# Was passiert wenn ich diesen Code ausführe?

2. Auflösen



```
public static int sum(int n) {
    if (n == 0) {
        return 0;
    }
    return sum(n-1) + n;
}
```

Fragen?

# Rekursion in Java – Aufgabe

Implementiere eine **statische, rekursive Methode** `IsPalindrome`, die:

- Einen String akzeptiert
- Einen **boolean** zurückgibt, ob der gesamte String ein Palindrom ist.

**Basisfall der Rekursion:**

- Wenn die String-Länge 0 oder 1 beträgt, gib `true` zurück

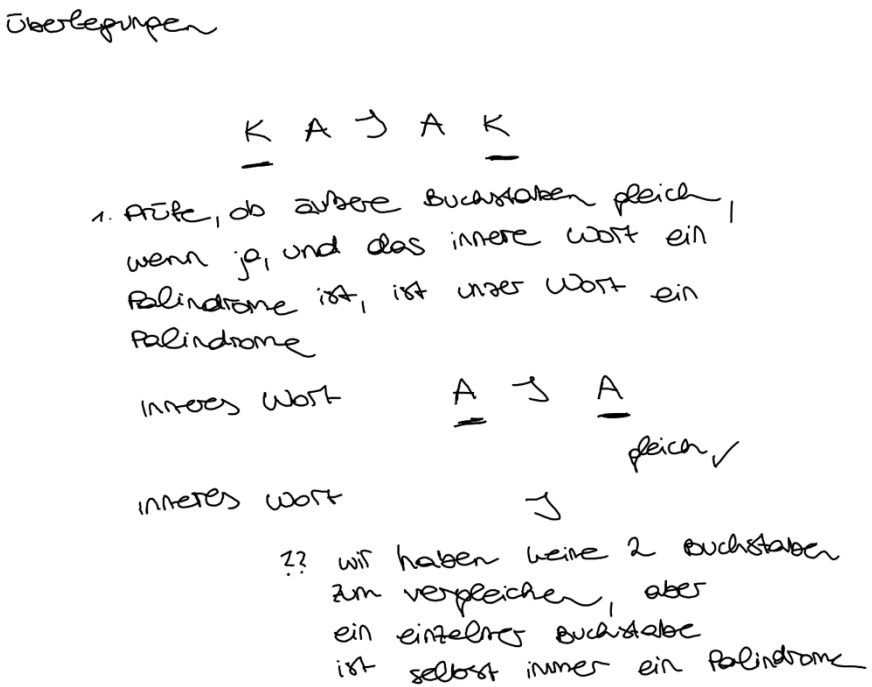
**Rekursiver Fall:**

- Nimm das **erste** und **letzte** Zeichen des Strings.
- Überprüfe, ob sie **gleich** sind.
- Verknüpfe das Ergebnis logisch mit einem Aufruf von `IsPalindrome` für den Teilstring ohne das erste und letzte Zeichen.

**Tipp:** Benutze `str.charAt(i)`, `str.substring(i, j)`

# Palindrome, Überlegungen

# Palindrome, Überlegungen



# Rekursion in Java – Lösung

```java
public class Palindrome {
    public static boolean isPalindrome(String word) {
        if (word.length() <= 1)
            return true;
        boolean firstAndLast = word.charAt(0) == word.charAt(word.length() - 1);
        String withoutFirstAndLast = word.substring(1, word.length() - 1);
        return firstAndLast && isPalindrome(withoutFirstAndLast);
    }
}
```

# Steigende Ziffern

`boolean digitsIncreasing(int n)`

Geben Sie zurück, ob die Ziffern von `n` mit $n \geq 0$ streng aufsteigend sind, wenn sie von der letzten zur ersten Ziffer gelesen werden.

**Beispiele:** `9631` $\rightarrow$ `true` (rückwärts gelesen: $1 < 3 < 6 < 9$), `21` $\rightarrow$ `true`, `7` $\rightarrow$ `true`, `0` $\rightarrow$ `true`, `1234` $\rightarrow$ `false`, `9931` $\rightarrow$ `false`, `59631` $\rightarrow$ `false`.

**Hinweis:** Vergleichen Sie die letzte mit der vorletzten Ziffer.

cs.rohlik.net -> EProg Übungsstunden -> Woche 6
Hint: die vorletzte Ziffer bekommen wir mit (n / 10) % 10

# Aufgabe 2: Arrays

1. Implementieren Sie die Methode `ArrayUtil.zeroInsert(int[] x)` in der Datei "ArrayUtil.java". Die Methode nimmt einen Array `x` als Argument und gibt einen Array zurück. Der zurückgegebene Array soll die gleichen Werte wie `x` haben, ausser: Wenn eine positive Zahl direkt auf eine negative Zahl folgt oder wenn eine negative Zahl direkt auf eine positive Zahl folgt, dann wird dazwischen eine 0 eingefügt.

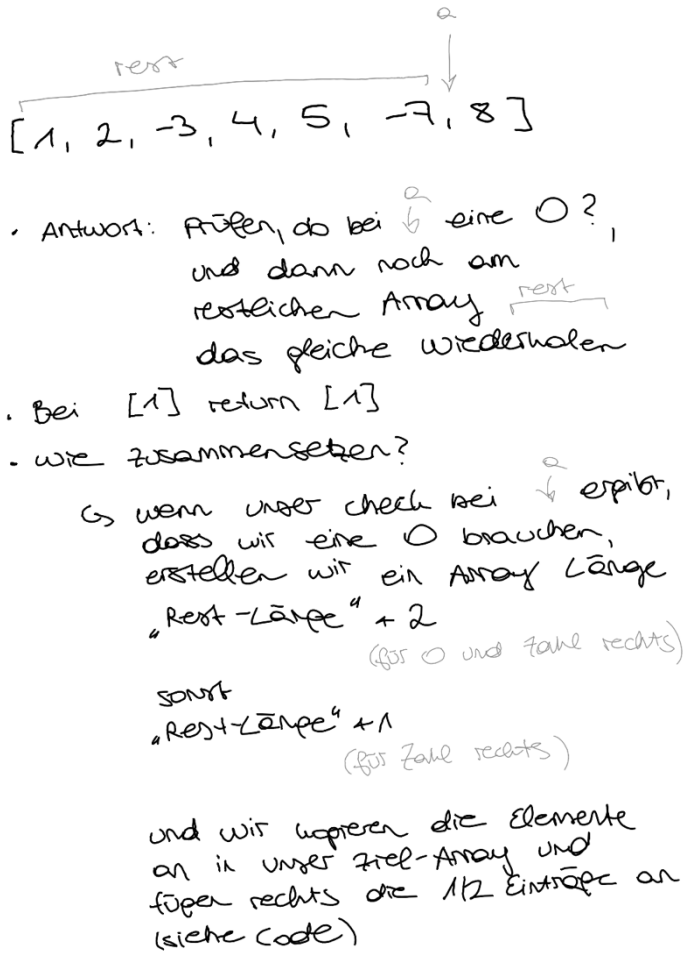
   Beispiele:

   - Wenn `x` gleich `[3, 4, 5]` ist, dann wird `[3, 4, 5]` zurückgegeben.
   - Wenn `x` gleich `[3, 0, -5]` ist, dann wird `[3, 0, -5]` zurückgegeben.
   - Wenn `x` gleich `[-3, 4, 6, 9, -8]` ist, dann wird `[-3, 0, 4, 6, 9, 0, -8]` zurückgegeben.

   Versuchen Sie, die Methode rekursiv zu implementieren.

-> siehe Tafel und Code

# Überlegungen zu Arrays



mehr Rekursion
nächste Woche

# Logisches Schliessen I

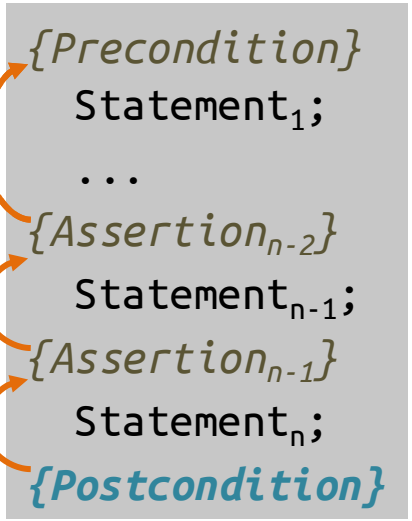
mehr dazu am Freitag in der Vorlesung und auch nächste Woche

# Stärkere und schwächere Aussagen

- Wir können *stärkere* und *schwächere* Aussagen unterscheiden
  - Wenn $P_1 \implies P_2$ gilt, dann ist $P_1$ stärker als $P_2$ (bzw. $P_2$ schwächer als $P_1$)

- Die stärkste Aussage ist `false`, da `false` alles impliziert
- Die schwächste Aussage ist `true`, da `true` nur `true` impliziert

# Rückwärtsschliessen: Vorgehen




- **Start:** Wählen (wissen, raten) einer sinnvollen *Nachbedingung*
- **Schrittweise:** Herleiten der vorherigen Aussage (Assertion$_{i-1}$) durch Einbeziehen des *Effekts* der vorherigen Anweisung (Statement$_i$)
- **Ziel:** Herleiten einer *notwendigen und hinreichenden Vorbedingung*

- Rückwärts = «Welche Vorbedingung braucht mein Code, damit er die gewählten Garantien (Nachbedingung) geben kann?»

# Weakest Precondition

- Die **schwächste Vorbedingung (weakest precondition)** ist die schwächste Vorbedingung, die die Postcondition impliziert.
- Die vorgestellten Regeln fürs Rückwärtsschliessen ergeben direkt die schwächsten Vorbedingungen.

# Weakest Precondition – Einfaches Beispiel

{ }

x = y*y ————

{x>4}

# Weakest Precondition – Einfaches Beispiel

{y*y>4}

x = y*y ————

# Weakest Precondition – Einfaches Beispiel

$\{|y|>2\}$

$x = y*y$ ————

# Weakest Precondition – Zweites Beispiel

{ }

y = x+3 ———

z = y+1 ———

{z>4}

# Weakest Precondition – Zweites Beispiel

$\{x+3+1 > 4\}$

$y = x+3$ ——————

$z = y+1$ ——————

# Weakest Precondition – Zweites Beispiel

$\{x > 0\}$

$y = x+3$ ——————

$z = y+1$ ——————

# Weakest Precondition - Beispiel

```
{              }

  x = a - 4;

{x>0}
```

# Weakest Precondition - Beispiel

```
{ a-4 > 0 }

  x = a - 4;

{x>0}
```

# Weakest Precondition - Beispiel

```
{ a > 4 }

  x = a - 4;

{x>0}
```

# Weakest Precondition - Beispiel

```
{ y-5 < 0 }

  x = y + z;

  y = y - 5;

{y < 0}
```

# Weakest Precondition - Beispiel

```
{ y < 5 }

  x = y + z;

  y = y - 5;

{y < 0}
```

# Weakest Precondition– Prüfungsbeispiel

```
{                    }

  w = a

  x = w + b;

  y = x * 2;

{y > 0 && b > 10}
```

# Weakest Precondition– Prüfungsbeispiel

```
{ (a+b)*2 > 0 && b > 10 }

  w = a
  x = w + b;
  y = x * 2;

{y > 0 && b > 10}
```

# Weakest Precondition– Prüfungsbeispiel

```
{ a > -b && b > 10 }

  w = a
  x = w + b;
  y = x * 2;

{y > 0 && b > 10}
```

# Weakest Precondition– Prüfungsbeispiel

```
{                    }

 p = 3 * q

 p = p + 1;

{p > 15}
```

# Weakest Precondition– Prüfungsbeispiel

```
{ 3*q + 1 > 15 }

 p = 3 * q

 p = p + 1;

{p > 15}
```

# Weakest Precondition– Prüfungsbeispiel

```
{ q > 14/3 }

 p = 3 * q

 p = p + 1;

{p > 15}
```

# Kahoot

https://create.kahoot.it/details/kahoot-u6/1dddb85b-38ac-46da-89aa-5da69a9c62e5?drawer=

# Vorbesprechung

# Aufgabe 1: Präfixkonstruktion



Gegeben seien zwei Strings `s` und `t` und ein Integer `n` mit $n \geq 0$. Schreiben Sie ein Programm, das zurückgibt, ob `s` eine Konkatenation von maximal `n` vielen Präfixen von `t` ist.

**Beispiele:**

- `s = "abcababc"`, `t = "abc"`, `n = 4`: Das Programm sollte `true` zurückgeben, da `"abc"` und `"ab"` Präfixe von `t` sind und `s` eine Konkatenation von `"abc"`, `"ab"`, `"abc"` ist.
- `s = "abcbcabc"`, `t = "abc"`, `n = 4`: Das Programm sollte `false` zurückgeben, da `"bc"` kein Präfix von `t` ist.
- `s = "abab"`, `t = "abac"`, `n = 2`: Das Programm sollte `true` zurückgeben, da `"ab"` ein Präfix von `t` ist und `s` eine Konkatenation von `"ab"`, `"ab"` ist.

Implementieren Sie die Methode `isPrefixConstruction(String s, String t, int n)` in der Klasse `PrefixConstruction`. Die Methode hat drei Argumente: die beiden Strings `s` und `t` und der Integer `n`. Sie dürfen davon ausgehen, dass der Integer grösser oder gleich 0 ist. In der Datei "PrefixConstructionTest.java" finden Sie Tests.

**Tipp:** Lösen Sie die Aufgabe rekursiv.

sehr schwer…

# Aufgabe 2: Weakest Precondition

Bitte geben Sie für die folgenden Programmsegmente die schwächste Vorbedingung (weakest precondition) an. Bitte verwenden Sie Java-Syntax. Alle Anweisungen sind Teil einer Java Methode. Alle Variablen sind vom Typ `int` und es gibt keinen Overflow.

1.
```
P: { ?? }
S:   m = n * 4;  k = m - 2;
Q: { n > 0 && k > 5 }
```

2.
```
P: { ?? }
S:   m = n * n;  k = m * 2;
Q: { k > 0 }
```

3.
```
P: { ?? }
S:   y = x + 3; z = y + 1;
Q: { z > 4 }
```

4.
```
P: { ?? }
S:   y = x + 1; z = y - 3;
Q: { z == 10 }
```



-> Moodle

# Aufgabe 3: Wörter Raten



Das Programm "WoerterRaten.java" enthält Fragmente eines Rate-Spiels, welches Sie vervollständigen sollen. In dem Spiel wählt der Computer zufällig ein Wort $w$ aus einer Liste aus und der Mensch muss versuchen, das Wort zu erraten. In jeder Runde kann der Mensch eine Zeichenfolge $z$ (welche einen oder mehrere Buchstaben enthält) eingeben und der Computer gibt einen Hinweis dazu. Folgende Hinweise sind möglich:

1. $w$ beginnt mit $z$
2. $w$ endet mit $z$
3. $w$ enthält $z$
4. $w$ enthält nicht $z$

```
Tipp? e
Das Wort enthält nicht "e"!
Tipp? a
Das Wort endet mit "a"!
Tipp? j
Das Wort beginnt mit "j"!
Tipp? v
Das Wort enthält "v"!
Tipp? java
Das Wort ist "java"!
Glückwunsch, du hast nur 5 Versuche benötigt!
```

# Aus Textdatei lesen

*woerter.txt*

```
8
wort
blasinstrument
computer
schlange
java
programmieren
welt
sugus
```

*Code zum Einlesen der Wörter:*

```java
Scanner scanner = new Scanner(new File("woerter.txt"));
int numWorter = scanner.nextInt();
String[] woerter = new String[numWorter];
for (int i = 0; i < woerter.length; i++) {
    woerter[i] = scanner.next();
}
```

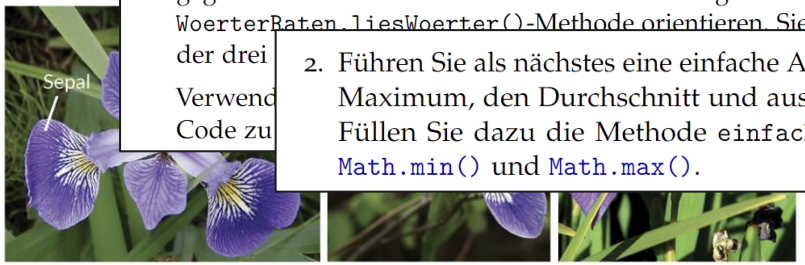
## Relativer Pfad

- ist relativ zum aktuellen Verzeichnis (“working directory”) des Programms (bei uns der Projektordner)

# Aufgabe 4: Datenanalyse

In dieser Aufgabe werden Sie die Kelchblattlänge von Iris Blumen, welche auch Schwertlilien genannt werden, analysieren. Dazu verwenden Sie ein öffentlich zugängliches Dataset [1] welches die Längen vom Kelchblatt (Sepal Length) von jeweils 150 verschiedenen Iris Blumen enthält. Es gibt sehr viele Arten von Iris Blumen, insgesamt sind 285 Arten bekannt. Diese zu unterscheiden ist ein komplex
Arten von Iris B
uns auf die folg

- Iris setosa
- Iris versic
- Iris virgin




Iris Versicolor | Iris Setosa | Iris Virginica

1. Das Programm soll als erstes die Kelchblattgrössen für die drei Arten von Iris Blumen aus den Dateien “sepal_length_setosa.txt”, “sepal_length_versicolor.txt”, und “sepal_length_virginica.txt” im Projekt-Verzeichnis in ein Array einlesen. Die Dateien haben ein ähnliches Format wie die “woerter.txt”-Datei der letzten Aufgabe. Die Kelchblattgrössen liegen als ganze Zahlen in Millimetern [mm] vor. Die erste Zahl gibt die Anzahl Daten im File an. Implementieren Sie dazu die Methode `liesLaengen()`, indem Sie die nötigen Daten aus dem gegebenen `Scanner` auslesen. Falls Sie Schwierigkeiten dabei haben, können Sie sich an der `WoerterRaten.liesWoerter()`-Methode orientieren. Sie können Ihre Methode anhand eines der drei
Verwend
Code zu

2. Führen Sie als nächstes eine einfache Analyse der Daten durch, indem Sie das Minimum, das Maximum, den Durchschnitt und ausserdem die
Füllen Sie dazu die Methode `einfacheAnalyse(`
`Math.min()` und `Math.max()`.

3. Die drei Werte, die Sie in 2. berechnet haben, sagen nicht viel über die Daten aus. Um die Daten besser zu verstehen, soll Ihr Programm ein Histogramm berechnen und auf der Konsole ausgeben. Die Textausgabe könnte ungefähr so aussehen:

```
[40,43)
[43,46)
[46,49)
[49,52) |
[52,55)
[55,58) ||
[58,61) ||||||
[61,64) ||||||||||
[64,67) |||||||||
[67,70) ||||||||||
[70,73) ||||
[73,76) ||
[76,79) |||||
[79,82) |
[82,85)
```

Implementieren Sie die Methode `histogrammAnalyse()`. Diese Methode soll zuerst den Benutzer nach der Anzahl der Histogramm-Klassen fragen, dann das Histogramm berechnen und schliesslich ausgeben. Zwei (leere) Methoden sind schon vorgegeben: `erstelleHistogramm()` und `klassenBreite()`. Für diese beiden Methoden existieren Tests in “DatenAnalyseTest.java” und Kommentare, welche Ihnen beim Schreiben des Programms helfen. Überlegen Sie sich, wie Sie das Problem weiter aufteilen möchten, und erstellen Sie entsprechende Methoden.

# Bonusaufgabe



## Aufgabe 5: Matrix (Bonus!)

**Achtung:** Diese Aufgabe gibt Bonuspunkte (siehe "Leistungskontrolle" im `www.vvz.ethz.ch`). Die Aufgabe muss eigenhändig und alleine gelöst werden. Die Abgabe erfolgt wie gewohnt per Push in Ihr Git-Repository auf dem ETH-Server. Verbindlich ist der letzte Push vor dem Abgabetermin. Auch wenn Sie vor der Deadline committen, aber nach der Deadline pushen, gilt dies als eine zu späte Abgabe. Bitte lesen Sie zusätzlich die allgemeinen Regeln.

Sie können davon ausgehen, dass der Parameter `matrix` nie den Wert `null` hat und nie auf eine $m \times n$ Matrix mit $m \leq 2$ oder $n \leq 2$ verweist. Sie können auch davon ausgehen, dass die Summe der Nachbarn sich ohne Overflow oder Underflow berechnen lässt.

# Danke!